

Report

of

AI-Powered Creative Generation Platform for Jewellery Marketing

Submitted To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Roll Number of Student: **CS-2341057**

Name of Student: **KUNAL SHARMA**

Batch: **2026–27**

Organisation of Internship:

Intelytica Business Analytics Pvt. Ltd.

Hyderabad, Telangana

1 Candidate's Declaration

I, **Kunal Sharma**, do hereby declare that the internship report titled “**AI-Powered Creative Generation Platform for Jewellery Marketing**” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree or diploma requirement, and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: Kunal Sharma

Roll No.: CS-2341057

2 Acknowledgement

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work. I express my gratitude to my parents for their blessings.

Signature

Student Name: Kunal Sharma

Roll No.: CS-2341057

Table of Contents

1	Candidate's Declaration	i
2	Acknowledgement	ii
3	Project Description	1
3.1	Introduction	1
3.2	Organization Profile	2
3.3	Problem Statement	3
3.4	Project Objectives	4
3.5	Scope of the Project	5
3.6	Technologies and Tools Used	6
3.7	System Architecture	8
3.8	Methodology	10
3.9	Expected Outcomes	11
3.10	Certificates of Completion and Other Communication Mail Proof	13
4	Bibliography	14

List of Figures

- 3.1 Layered architecture of the Prism platform. The frontend (Layer 1) communicates with the FastAPI REST layer (Layer 2), which orchestrates the agent layer (Layer 3) and the service layer (Layer 4). External AI providers supply text, image and video generation, while PostgreSQL and storage handle persistence. 8
- 3.2 End-to-end creative generation workflow. A one-line campaign brief is expanded by Claude into a rich brief, market intelligence and strategy are produced in parallel, ready-to-use copy is generated, the user uploads a jewellery photograph which Claude Vision describes, five scene-by-scene reel scripts are produced, scene images are generated by the chosen provider with jewellery-identity locks, and the final package is optionally animated into a video and exported to six social sizes. . . . 10

Note: Figure 3.1 (System Architecture) and Figure 3.2 (Creative Generation Workflow) are original diagrams prepared by the author to illustrate the Prism platform’s layered design and end-to-end pipeline. All other visual artefacts referenced in the report are illustrative descriptions of components shipped in the production codebase.

3 Project Description

3.1 Introduction

The year 2026 has been a watershed moment for applied generative artificial intelligence. Large language models have matured from research artefacts into dependable production primitives, vision-language models can now describe an image with the precision of a trained cataloguer, and multi-modal text-to-image and text-to-video systems have crossed the threshold of commercial viability for retail marketing workloads. What was, only eighteen months ago, an experimental capability confined to a handful of well-funded studios is today a routine part of the marketing technology stack of any mid-sized consumer brand. The opportunity this creates for the unorganised retail sector in India, and specifically for jewellery retailers who must produce high-quality creative content for every festival on the calendar, is enormous.

The Indian jewellery market is uniquely demanding. It is intensely local, deeply tied to festival cadences — Diwali, Raksha Bandhan, Akshaya Tritiya, Karwa Chauth, Dhanteras — and characterised by repeat purchase cycles where the same customer expects to be re-engaged several times a year with fresh, occasion-relevant creative. A single retail store may need a new Instagram carousel, a WhatsApp broadcast, a catalogue narrative, a Meta ad set, and a fifteen-second reel every two weeks throughout the festive season. Traditional creative agencies, priced for monthly retainers and turnaround cycles measured in days, simply cannot keep pace with this cadence. The gap between what the market demands and what agencies can supply is the central problem the Prism platform was built to close.

This report documents the work I undertook during a three-month internship, from 27 May 2026 to 27 August 2026, at **Intelytica Business Analytics Pvt. Ltd.**, Hyderabad, in the capacity of a Data Science and AI Intern. I was assigned to the engineering team building *Prism*, a generative AI marketing platform for jewellery retailers, branded in its lead deployment as the JD Solitaires Marketing Platform. The platform’s defining capability is the conversion of a one-line campaign brief — for example, “Diwali necklace push” — into a complete, ready-to-post creative package comprising market intelligence, strategy, channel-specific copy, scene-by-scene reel scripts, generated product imagery, and short-form video, all in a matter of minutes rather than days. Crucially, the system preserves the identity of the uploaded jewellery piece through every AI generation, so that the actual product — its metal, gemstones, proportions and design — is what the customer sees in the final creative, and not a redesigned approximation.

The remainder of this report is organised as follows. Section 3.2 profiles the host organisation. Section 3.3 articulates the problem statement in detail. Section 3.4 enumerates the project objectives. Section 3.5 defines the scope. Section 3.6 lists the technologies and tools used. Section 3.7 describes the system architecture. Section 3.8

explains the methodology adopted. Section 3.9 documents the expected outcomes and the intern's specific contributions. Section 3.10 lists the certificates.

3.2 Organization Profile

Intelytica Business Analytics Pvt. Ltd. is a Hyderabad-based technology firm headquartered at Somajiguda, Telangana, that builds data-science and artificial-intelligence products for retail and consumer brands. The company operates at the intersection of business analytics, machine learning engineering, and applied generative AI, with a particular focus on the Indian domestic retail sector. Its commercial offering spans customer analytics, marketing-resource-management platforms, and bespoke AI agent workflows that translate business problems into deployable decision-support systems.

The Data Science and AI department, where this internship was hosted, is responsible for the design, development and operation of the company's flagship generative AI marketing platform, Prism. The department is structured as a small, cross-functional product engineering team comprising backend engineers working in Python and FastAPI, frontend engineers working in React and TypeScript, AI/ML engineers responsible for model integration and prompt engineering, and a product lead who sets the creative and commercial direction of the platform. Interns are inducted into the same engineering rhythm as full-time engineers: daily stand-ups, weekly sprint planning, code reviews via pull requests, and a structured mentorship cadence with a designated industry mentor.

The internship was conducted in a hybrid working model, with the office located in Hyderabad and a portion of the work performed remotely. Working hours were 10:00 AM to 6:00 PM, Monday to Friday, and the duration was three calendar months. The terms of the engagement, including a monthly stipend of ₹8,000, were formalised in an offer letter bearing reference number **INTELYTICA/HR/INTERN/2026/003**, dated 20 May 2026, and signed by the Director, Mr. Amitendra Kumar. The offer letter is reproduced as a physical appendix to this report. The company maintains a strict policy on data confidentiality and intellectual property: all code, models, documents and materials produced during the internship are the sole property of Intelytica Business Analytics Pvt. Ltd., and no client-identifying, commercially sensitive, or personally identifiable information is shared outside the organisation.

The cultural environment of the team was that of a modern product startup operating inside a stable parent organisation. Decisions were evidence-driven, experiments were run quickly and discarded just as quickly if they failed to deliver measurable improvements, and the engineering culture placed a high premium on readable code, sensible abstractions, and honest post-mortems. This environment shaped both the technical work I produced and the professional habits I take away from the internship.

3.3 Problem Statement

The jewellery retail vertical in India presents a marketing problem that is unusually well-suited to generative AI and unusually poorly served by the existing creative-supply chain. The problem can be decomposed into four interlocking sub-problems, each of which the Prism platform was explicitly designed to address.

The first sub-problem is **cadence**. A jewellery retailer operating in a Tier-1 Indian city typically services eight to ten festival peaks in a calendar year, each requiring a refresh of Instagram carousels, WhatsApp broadcast copy, catalogue narrative, Meta ad creatives, and short-form reel video. The traditional agency model, priced for monthly retainers and turnaround times of three to five business days per creative set, structurally cannot keep pace with this rhythm. The result is that retailers either underinvest in creative freshness and lose share of voice during peak demand windows, or overspend on agencies and erode margin. Neither outcome is acceptable. The platform must therefore be capable of producing a complete, brand-coherent creative package in minutes, not days.

The second sub-problem is **product identity**. Generic text-to-image models, when prompted with “a woman wearing a gold necklace”, will happily produce an aesthetically pleasing image of a woman wearing a gold necklace — but it will be a *different* necklace every time, and almost never the exact piece the retailer actually sells. For a category where the product itself is the hero and the customer’s purchase decision hinges on a specific design, this is a fatal flaw. The system must therefore preserve the identity of an uploaded jewellery photograph through every downstream generation — the metal, the gemstones, the proportions, the design language — without allowing the AI to redesign it.

The third sub-problem is **creative-director judgement**. The marketing teams at jewellery retail stores are excellent at merchandising and customer relationships, but they are not trained creative directors. Asking them to specify lighting, composition, mood, cultural targeting and channel-appropriate copy from a blank page produces inconsistent output. The system must therefore encode, in software, the creative-director judgement that would otherwise have to be hired from an agency — brand voice, size discipline, occasion theming, channel-specific formatting, and culturally appropriate localisation for the North-Indian domestic market rather than for the NRI or Western audience that most foundation models default to.

The fourth sub-problem, which is the one this internship most directly addressed, is **multi-provider orchestration with consistency**. No single image or video model is the right answer for every creative need. OpenAI’s gpt-image-2 offers the highest identity fidelity for reference-image editing, xAI’s Grok produces stylistically distinctive scenes, Google’s Gemini Flash Image handles size calibration gracefully via system instructions, and Google’s Veo 3.1 produces the highest-quality short video clips. The system must

therefore orchestrate these providers behind a single coherent interface, with prompt engineering that adapts to each provider’s strengths and weaknesses, while maintaining continuity of person, outfit, lighting and product identity across scenes generated by different providers in sequence.

3.4 Project Objectives

The objectives of the project, as scoped at the beginning of the internship and refined through the first sprint, were as follows.

Objective 1: Integrate Anthropic Claude as the text and vision backbone. The platform needed a single, dependable large language model to drive every text-generation agent — the brief expander, the trend agent, the strategy agent, the creative copy agent, and the reel-script agent — and to serve as the connective vision tissue through jewellery description, dimension estimation, occasion-theme authoring, and result captioning. The objective was to wrap the Anthropic API in a single internal gateway function with retry, back-off, token-usage tracking and structured-JSON repair, so that downstream agents could call it without having to know anything about the underlying provider.

Objective 2: Build a Vision-based jewellery identity and dimension estimation pipeline. The system had to be able to ingest an uploaded jewellery photograph and produce a dense, set-aware description of the piece — every component of a matched set described, not just the largest, with estimated real-world sizes — and an occasion-aware theme block that could be injected into every downstream image prompt. This description is the single source of truth for product identity throughout the pipeline.

Objective 3: Implement a multi-provider image generation service. The platform needed to support four image providers — OpenAI gpt-image-2, xAI Grok, Google Gemini Flash Image, and Google Imagen 4 — behind a unified interface, each invoked with provider-specific identity-preservation techniques. OpenAI required an `input_fidelity` set to high, deliberate image ordering, indexed multi-image prompting and an optional two-pass generation for worn jewellery. Grok required reference-image editing with a strict identity block. Gemini required size calibration as a separate system instruction. Imagen 4 supported text-to-image with aspect ratio and negative prompts but no reference edit. The objective was to abstract these differences behind a single dispatch surface.

Objective 4: Build the video generation pipeline. Two independent video paths were in scope: a Grok text-to-video path producing three five-second clips stitched into a single fifteen-second reel with a character anchor that preserves the same person across clips, and a Google Veo 3.1 path producing a single eight-second jewellery advertisement from either a photograph, a reel script, or a generated scene image, with fast and quality tiers.

Objective 5: Engineer a layered, priority-ordered prompt system. The prompt builder had to assemble, in a strict priority order: a mandatory scene block, an outfit

lock, a jewellery identity block conditioned on the scene’s jewellery state, a size anchor, a continuity note, a style base, a person style, prop discipline rules, an occasion atmosphere block, a negative prompt, and an edit-correction block when applicable. The objective was to ensure that no downstream style instruction could override the scene’s mandatory content, and that the product identity was always preserved.

Objective 6: Maintain brand, product and visual consistency across scenes. The system had to keep the same woman, outfit and lighting across scenes generated by potentially different providers, through character anchors for video and previous-scene chaining for images, and to enforce brand voice through constant style bases and an explicit avoid-and-prefer word list.

Objective 7: Add post-processing for social distribution. A complete creative package requires that generated images be resized to six standard social-media export sizes, optionally composited with a product-inset badge that overlays the actual uploaded photograph pixel-exactly as a corner element, and made available for download through time-limited presigned URLs.

3.5 Scope of the Project

The scope of the internship was deliberately bounded to the **Creative Generation module** of the Prism platform, with primary focus on the Quick Analysis fast-lane that bypasses the gated approval pipeline. This module is the heart of the product — the subsystem that converts a one-line campaign brief and an uploaded jewellery photograph into the complete set of creative assets described in the introduction — and it was the module where the intern’s AI/ML and generative-services contributions could have the highest leverage.

In scope were the following components and responsibilities: (a) the Anthropic Claude integration layer, including the gateway function with retry, back-off and token tracking, and the structured-JSON repair pipeline; (b) the agent layer comprising the brief expander, trend, strategy, creative and reel-script agents, all of which call Claude through the gateway; (c) the Vision helpers for jewellery description, dimension estimation, occasion-theme authoring and scene captioning; (d) the four image-generation service modules (OpenAI, Grok, Gemini and Imagen); (e) the two video-generation service modules (Grok reel and Veo jewellery ad); (f) the prompt-engineering brain, including the layered prompt builder, the size-system-instruction generator, and the jewellery-state detector; (g) the post-processing helpers for resize, social-size export and product-inset compositing; and (h) the integration of these services into the FastAPI route layer that exposes them as REST endpoints.

Out of scope for this internship, but actively owned by other members of the team, were: the React 18 frontend application and its Material-UI component library, which was built and maintained by the frontend engineers; the gated Research-Strategy-Creative pipeline with its approval gates and StageRun versioning, which was maintained by a

senior backend engineer; the infrastructure and DevOps layer, including Docker composition, deployment scripts and logging infrastructure; and the integration with payment, billing and cost-estimation services. The intern collaborated closely with the owners of these out-of-scope components — the frontend engineers consumed the REST endpoints I produced, the senior backend engineer reviewed my pull requests, and the DevOps engineer helped diagnose production issues — but did not own them.

This scoping reflects the reality of a team-based product engineering environment. The intern’s contributions are reported honestly in Section 3.9, with explicit attribution of which components were authored individually, which were co-developed with peers, and which were the work of the wider team that the intern integrated against.

3.6 Technologies and Tools Used

The Prism platform is built on a modern, polyglot stack that pairs a Python backend with a React frontend and an external layer of foundation-model APIs. The technologies used during the internship, with their roles and approximate versions, are summarised in Table 3.1.

Table 3.1. Technologies and tools used in the Prism platform.

Layer	Technology	Role in the platform
Backend language	Python 3.11	Application code for agents, services, routes.
Web framework	FastAPI	REST API surface, dependency injection, async.
ASGI server	Uvicorn	Serves the FastAPI application in production.
ORM	SQLAlchemy 2.0 (typed)	Database models with Mapped[...] annotations.
Database	PostgreSQL (JSONB)	Persistent state; JSONB columns for agent outputs.
Migrations	Alembic	Schema versioning and migration management.
LLM	Anthropic Claude (sonnet-4-6)	Text generation, Vision (describe, caption, theme).
Image AI	OpenAI gpt-image-2	Default image provider; reference-image editing.
Image AI	xAI Grok (grok-imagine-image)	Stylistically distinctive alternative provider.
Image AI	Google Gemini Flash Image	Size-calibration via system instruction.
Image AI	Google Imagen 4	Text-to-image with aspect ratio and negatives.

Layer	Technology	Role in the platform
Video AI	xAI Grok video	Three five-second clips stitched into a reel.
Video AI	Google Veo 3.1	Jewellery advertisement (fast and quality tiers).
Media processing	Pillow	Resize, contain-and-pad, product-inset compositing.
Media processing	ffmpeg (-ccopy)	Stream-copy stitching of Grok video clips.
Storage	Local / S3 (factory pattern)	Pluggable binary asset storage with same interface.
Auth	JWT bearer + bcrypt	Authentication and password hashing.
Frontend (team)	React 18 + TypeScript + Vite	Single-page application consuming the REST API.
Frontend UI (team)	Material UI	Component library for the creative studio surface.

The rationale for the principal choices is as follows. **FastAPI** was selected over Flask or Django because the platform’s endpoints are overwhelmingly JSON-in, JSON-out and benefit from Pydantic validation, automatic OpenAPI documentation, and native async support for the long-running background tasks that image and video generation require. **SQLAlchemy 2.0** with its new typed Mapped[. . .] annotations was chosen for its ability to express the rich, nested domain model — projects, briefs, decision inputs, runs, creative assets, audit events — in a way that survives refactoring with the help of the type checker, while still allowing JSONB columns for the semi-structured agent outputs that change frequently during product iteration.

PostgreSQL with JSONB is the keystone of the data model. Rather than normalising every agent output into its own table, the team chose to persist the entire creative_output envelope as a single JSONB column on the QuickAnalysisRun row. This decision traded query-time joins for write-time simplicity and schema flexibility, and was vindicated by the speed with which the team could iterate on agent output shapes without running a migration each time. The trade-off is that nested writes under concurrent background tasks require careful use of flag_modified and row locks, which the intern helped implement and document.

Anthropic Claude was chosen as the LLM backbone after a brief evaluation of alternatives, on the strength of its structured-output reliability, the quality of its Vision capabilities for fine-grained jewellery description, and the predictability of its tool-use behaviour. **The multi-provider image stack** reflects the team’s view that no single provider is universally best: OpenAI wins on identity fidelity, Grok on stylistic distinctiveness, Gemini on size calibration, and Imagen 4 on pure text-to-image with negative

prompts. Abstracting these behind a dispatch surface let the team swap providers per generation and A/B test stylistic outcomes.

3.7 System Architecture

The Prism platform follows a layered architecture in which a React single-page application communicates over REST with a FastAPI backend, which in turn orchestrates a layer of LLM agents, a layer of image and video generation services, and a persistence layer of PostgreSQL and pluggable storage. The architecture is depicted in Figure 3.1.

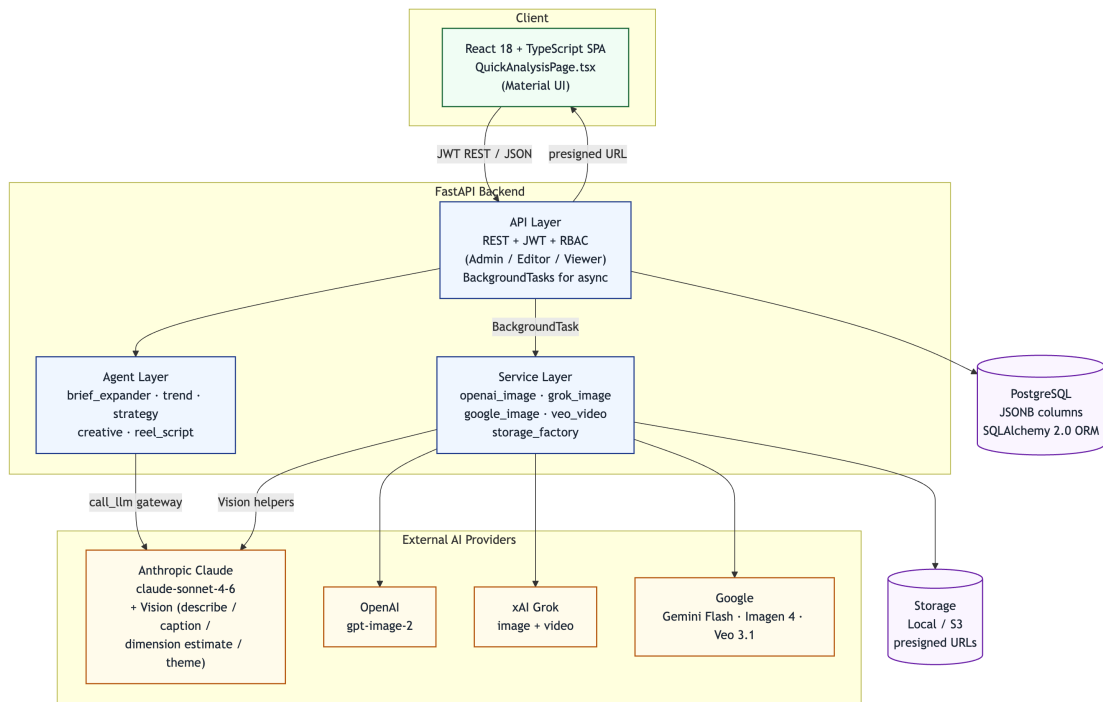


Figure 3.1. Layered architecture of the Prism platform. The frontend (Layer 1) communicates with the FastAPI REST layer (Layer 2), which orchestrates the agent layer (Layer 3) and the service layer (Layer 4). External AI providers supply text, image and video generation, while PostgreSQL and storage handle persistence.

The **frontend** is a React 18 single-page application built with TypeScript and Vite, using Material UI for the component library. The primary creative surface is the `QuickAnalysisPage` component, which hosts the campaign input, the results tabs (trends, strategy, creative copy), the jewellery upload form, the product-shot cards, the reel-script editor, the scene generator, the video panel and the export controls. The frontend was owned by the frontend engineering team; the intern's responsibility was to design the REST endpoints and the response shapes that this surface consumes.

The **API layer** is a FastAPI application served by Uvicorn. It exposes a REST surface under `/api/projects/{project_id}/quick-analysis` for the creative-generation module, alongside endpoints for authentication, project management, briefs and the gated pipeline. Authentication uses JWT bearer tokens, and role-based access control is enforced through FastAPI dependencies that resolve the current user into one

of three roles — admin, editor or viewer — with generation endpoints requiring editor privileges and read endpoints accepting viewer privileges. Long-running operations — scene generation, video generation — are dispatched to FastAPI’s in-process `BackgroundTasks` and return HTTP 202 with a nested status object that the frontend polls.

The **agent layer** comprises five LLM agents, each implemented as a Python module that calls the Claude gateway and parses the structured-JSON response: the brief expander, which turns a one-line occasion and emphasis into a rich campaign brief; the trend agent, which produces market intelligence; the strategy agent, which produces moodboard and channel strategy; the creative agent, which produces ready-to-use copy for Instagram, WhatsApp, catalogue and Meta ads; and the reel-script agent, which produces five scene-by-scene scripts each tagged with a jewellery-state enum that drives downstream image generation. The intern was responsible for the integration and prompt engineering of these agents, working from product direction provided by the senior team.

The **service layer** comprises four image-generation service modules (OpenAI, Grok, Google Gemini, Google Imagen), the Veo video service module, the storage factory with its local and S3 implementations, and the post-processing helpers for resize and inset compositing. This is the layer where the intern’s primary contributions were concentrated; the design and implementation of these services is described in detail in Section 3.9.

The **persistence layer** consists of PostgreSQL for relational and JSONB state, accessed through SQLAlchemy 2.0 with Alembic migrations, and a pluggable storage abstraction that selects between a local filesystem service in development and an S3 service in production. Binary assets — generated images and videos — are stored under a key scheme of `workspaces/{ws}/projects/{proj}/quick-visuals/{uuid}.{jpg|mp4}`, with metadata and 24-hour presigned download URLs persisted in the run’s `creative_output` JSONB column.

Data flow across the layers follows a consistent pattern. The frontend issues a REST request with a JWT bearer token; the API layer resolves the current user and project, dispatches the appropriate agent or service, and either returns the result synchronously or enqueues a background task and returns HTTP 202. Agents call the Claude gateway, which handles retry and token tracking. Services call the appropriate external AI provider, upload the resulting binary asset to storage, and write the storage key and presigned URL back into the run’s JSONB column. The frontend polls the run status until completion, then reads the nested status flags to display results.

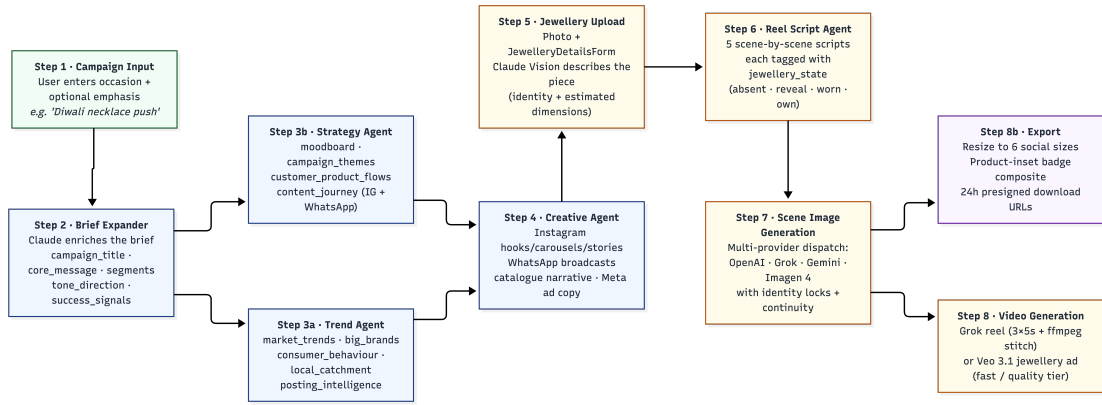


Figure 3.2. End-to-end creative generation workflow. A one-line campaign brief is expanded by Claude into a rich brief, market intelligence and strategy are produced in parallel, ready-to-use copy is generated, the user uploads a jewellery photograph which Claude Vision describes, five scene-by-scene reel scripts are produced, scene images are generated by the chosen provider with jewellery-identity locks, and the final package is optionally animated into a video and exported to six social sizes.

3.8 Methodology

The internship was conducted under an agile, research-driven methodology that combined weekly sprints with hypothesis-driven experimentation, structured code review, and continuous documentation. The methodology was inherited from the team’s existing engineering culture and was adapted by the intern to the specific demands of prompt engineering, where the design space is large, the feedback loop is short, and the right answer is rarely obvious from first principles.

Phase 1 — Discovery (Weeks 1–2). The first two weeks were spent reading the existing codebase end to end. The intern cloned the repository, set up the local development environment with Docker, ran the test suite, and walked through every module in the agent and service layers with the senior backend engineer. The deliverables of this phase were a written architecture summary, a list of identified technical-debt items, and a draft of the specific objectives (Section 3.4) that the intern proposed to take ownership of. This phase also included orientation on the company’s data-security and confidentiality policies, code-review conventions, and the pull-request workflow.

Phase 2 — Implementation (Weeks 3–7). The implementation phase was organised into two-week sprints. Each sprint began with a planning meeting in which the intern and the industry mentor agreed on a small set of objectives — typically two or three — that would be the focus of the next ten working days. Each day began with a stand-up in which the intern reported progress, blockers and plans. Each pull request was reviewed by at least one senior engineer before merge, and review feedback was treated as a learning opportunity rather than a critique. The intern’s implementation work in this phase concentrated on the image and video service modules and on the layered prompt builder.

Phase 3 — Refinement (Weeks 8–10). The refinement phase focused on prompt engineering and identity-preservation logic. The methodology here was explicitly hypothesis-driven: the intern would formulate a hypothesis (“adding an explicit SINGLE-INSTANCE rule to the jewellery identity block will reduce the rate at which the model generates duplicate pieces in a worn scene”), design a small evaluation corpus of representative inputs, run the generation pipeline against this corpus with and without the change, and measure the outcome. Results were documented in an internal experiment log and reviewed with the mentor before the change was merged or discarded. This phase also included hardening work on the JSON-repair pipeline, the size-calibration system instructions, and the two-pass OpenAI generation flow.

Phase 4 — Documentation (Weeks 11–12). The final two weeks were spent documenting the work, both in the codebase (docstrings, README updates, architecture notes) and in this internship report. The intern also participated in a knowledge-transfer session in which the implemented components were walked through for the benefit of the next intern cohort, and contributed to the internal knowledge base with a write-up of the layered prompt system and the multi-provider dispatch pattern.

Throughout all four phases, the team maintained a culture of honest post-mortems. When an experiment failed — and several did — the failure was documented alongside the hypothesis and the measurement, so that the next engineer approaching the same problem would not repeat the same dead end. This culture had a profound effect on the intern’s understanding of how engineering judgement is built: not from a string of unbroken successes, but from a curated sequence of failures, each of which narrowed the design space for the next attempt.

3.9 Expected Outcomes

This section documents the tangible outcomes of the internship, with explicit attribution of the intern’s individual contributions versus work that was co-developed with the team. The outcomes are presented in the order in which they appear in the creative-generation pipeline.

Outcome 1 — Layered prompt builder. The intern authored the `build_reel_scene_prompt` function, which assembles, in strict priority order, eleven distinct prompt layers: a mandatory scene block wrapped in explicit render guards; an outfit lock extracted from the scene text; a jewellery identity block conditioned on the scene’s jewellery state; a size anchor derived from a structured profile, a catalogue match, a per-type template, or a generic fallback; a continuity note; a style base specifying camera, lens, lighting and aesthetic; a person style, either narrative or default; conditional prop-discipline, phone-rule and non-festive-default rules; an occasion atmosphere block explicitly marked as additive rather than replacing; a merged negative prompt declared as overriding; and an edit-correction block with an ABSOLUTE JEWELLERY LOCK guard when applicable. The priority order ensures that no downstream style instruction

can override the scene’s mandatory content, and that the product identity is preserved even when the model is being asked to make a correction. This was the intern’s largest individual contribution.

Outcome 2 — Claude Vision integration. The intern integrated four Claude Vision helpers — `describe_jewellery_with_vision`, `estimate_dimensions_with_vision`, `get_occasion_theme_from_agent`, and `caption_image_with_vision` — into a coherent vision pipeline. The jewellery-description helper is set-aware, meaning it describes every component of a matched jewellery set rather than only the largest piece, and includes estimated real-world sizes that feed into the size anchor. The occasion-theme helper asks Claude to author a seven-point mandatory theme block (setting, lighting, clothing defaults, emotion and pose, must-include props, must-exclude items, and wearer gender) for any occasion, with a static fallback if the call fails. The image-captioning helper replaces the guessed caption with what is actually rendered, so that the displayed caption always matches reality.

Outcome 3 — Multi-provider image service abstraction. The intern co-developed, with the senior backend engineer, the four image-generation service modules (OpenAI, Grok, Gemini, Imagen 4) and the dispatch surface that selects between them. The intern’s individual contributions within this work were: the OpenAI two-pass generation flow, which renders a model with no jewellery in pass one and then edits the exact piece on in pass two with `input_fidelity` set to high and the jewellery image passed first, to avoid the reference-edit mangling that occurs when the jewellery is the sole canvas; the Gemini size-calibration system instruction, which elevates size discipline to a top-priority system instruction that the model treats as overriding its own instincts; and the Grok reference-image editing flow, which passes the uploaded photograph as a high-priority reference and falls back to text-only generation when the edits endpoint errors. The provider-abstraction layer, which would replace the inline `ifprovider==...` branches with a strategy-registry interface, was identified as a suggested improvement and is documented in the team’s technical-debt log.

Outcome 4 — Video generation pipelines. The intern implemented two video-generation paths. The Grok reel path reads the approved scene set, asks Claude Vision to describe the first scene into a character anchor, splits the reel narrative across three five-second clips each carrying the anchor, generates the clips sequentially to cap peak memory, downloads them, and stitches them with `ffmpeg-ccopy` (stream copy, no re-encode) before uploading the final MP4 to storage. The Veo jewellery-ad path supports three sources — photograph alone, photograph plus reel script, or a generated scene image — and selects the fast or quality tier based on the user’s choice, with a ten-minute hard timeout. Both paths report progress through nested status objects in the run’s JSONB column, and both write errors back using a fresh database session to avoid poisoned transactions.

Outcome 5 — Post-processing and social export. The intern implemented the post-processing helpers that take a generated image and produce a downloadable social-media package: resize to six standard social sizes via either a Grok re-frame call or a Pillow contain-and-pad fallback; composite a product-inset badge that overlays the actual uploaded photograph pixel-exactly as a corner element, so that the genuine piece is always shown next to the model; and generate 24-hour presigned download URLs. The resize path raises a custom `InsufficientCreditsError` that the API layer maps to HTTP 402 when the Grok credit pool is exhausted, and falls back gracefully to Pillow for non-native aspect ratios.

Outcome 6 — Documentation and knowledge transfer. Beyond the code, the intern produced comprehensive documentation: docstrings on every public function, an internal architecture note on the layered prompt system, a write-up of the multi-provider dispatch pattern for the team knowledge base, and a knowledge-transfer session for the next intern cohort. This internship report is itself part of the documentation outcome.

3.10 Certificates of Completion and Other Communication Mail Proof

The following documents are attached as physical appendices to this report and are available for inspection by the faculty supervisor and the internship evaluation committee.

- (a) **Offer Letter**, Ref. No. **INTELYTICA/HR/INTERN/2026/003**, dated 20 May 2026, signed by Mr. Amitendra Kumar, Director, Intelytica Business Analytics Pvt. Ltd., offering the intern the position of Data Science and AI Intern for a three-month period from 01 June 2026 to 31 August 2026, with a monthly stipend of ₹8,000 and the terms and conditions reproduced in Section 3.2 of this report.
- (b) **Internship Completion Certificate**, to be issued by Intelytica Business Analytics Pvt. Ltd. upon satisfactory completion of the internship duration, on the company's letterhead and signed by the Director. A reproduction of the certificate text is provided in the front matter of this report.

The intern affirms that all the documents attached are genuine and were obtained in the normal course of the internship engagement. Any queries regarding the authenticity or content of these documents may be addressed to the intern or, through the faculty supervisor, to the contact at Intelytica Business Analytics Pvt. Ltd. identified in the offer letter.

4 Bibliography

- [1] Anthropic, “Claude API Reference: Messages,” *Anthropic Documentation*, 2026. <https://docs.anthropic.com/en/api/messages>
- [2] Anthropic, “Claude Vision: Image and Document Understanding,” *Anthropic Documentation*, 2026. <https://docs.anthropic.com/en/docs/build-with-claude/vision>
- [3] OpenAI, “Image Generation API: gpt-image-2,” *OpenAI Platform Documentation*, 2026. <https://platform.openai.com/docs/api-reference/images>
- [4] OpenAI, “Images Edit Endpoint: input_fidelity Parameter,” *OpenAI Cookbook*, 2026.
- [5] xAI, “Grok Image and Video API,” *xAI Developer Documentation*, 2026. <https://docs.x.ai/docs>
- [6] Google, “Gemini API: Image Generation,” *Google AI for Developers*, 2026. <https://ai.google.dev/gemini-api/docs/image-generation>
- [7] Google, “Imagen 4: Text-to-Image Generation,” *Google Cloud Vertex AI Documentation*, 2026.
- [8] Google, “Veo 3.1: Video Generation API,” *Google Cloud Vertex AI Documentation*, 2026. <https://cloud.google.com/vertex-ai/generative-ai/docs/video>
- [9] Sebastián Ramírez et al., “FastAPI: Modern, Fast (high-performance) Web Framework for Building APIs with Python,” *FastAPI Documentation*, 2026. <https://fastapi.tiangolo.com>
- [10] Michael Bayer et al., “SQLAlchemy 2.0: The Database Toolkit for Python,” *SQLAlchemy Documentation*, 2026. <https://docs.sqlalchemy.org/en/20/>
- [11] PostgreSQL Global Development Group, “JSONB Data Type and Indexing,” *PostgreSQL Documentation*, 2026. <https://www.postgresql.org/docs/current/datatype-json.html>
- [12] Meta Platforms Inc., “React 18: Concurrent Features and Suspense,” *React Documentation*, 2026. <https://react.dev>
- [13] Wei, J. et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [14] Brown, T. et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [15] Rombach, R. et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [16] Park, J. et al., “Generative Agents: Interactive Simulacra of Human Behavior,” *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [17] LangChain Inc., “LangChain: A Framework for Developing Applications Powered by Language Models,” *LangChain Documentation*, 2026. <https://python.langchain.com>
- [18] LlamaIndex Inc., “LlamaIndex: Data Framework for LLM Applications,” *LlamaIndex Documentation*, 2026. <https://docs.llamaindex.ai>
- [19] Hunter, J. D., “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

- [20] FFmpeg Developers, “FFmpeg: A Complete, Cross-Platform Solution to Record, Convert and Stream Audio and Video,” *FFmpeg Documentation*, 2026. <https://ffmpeg.org>
-